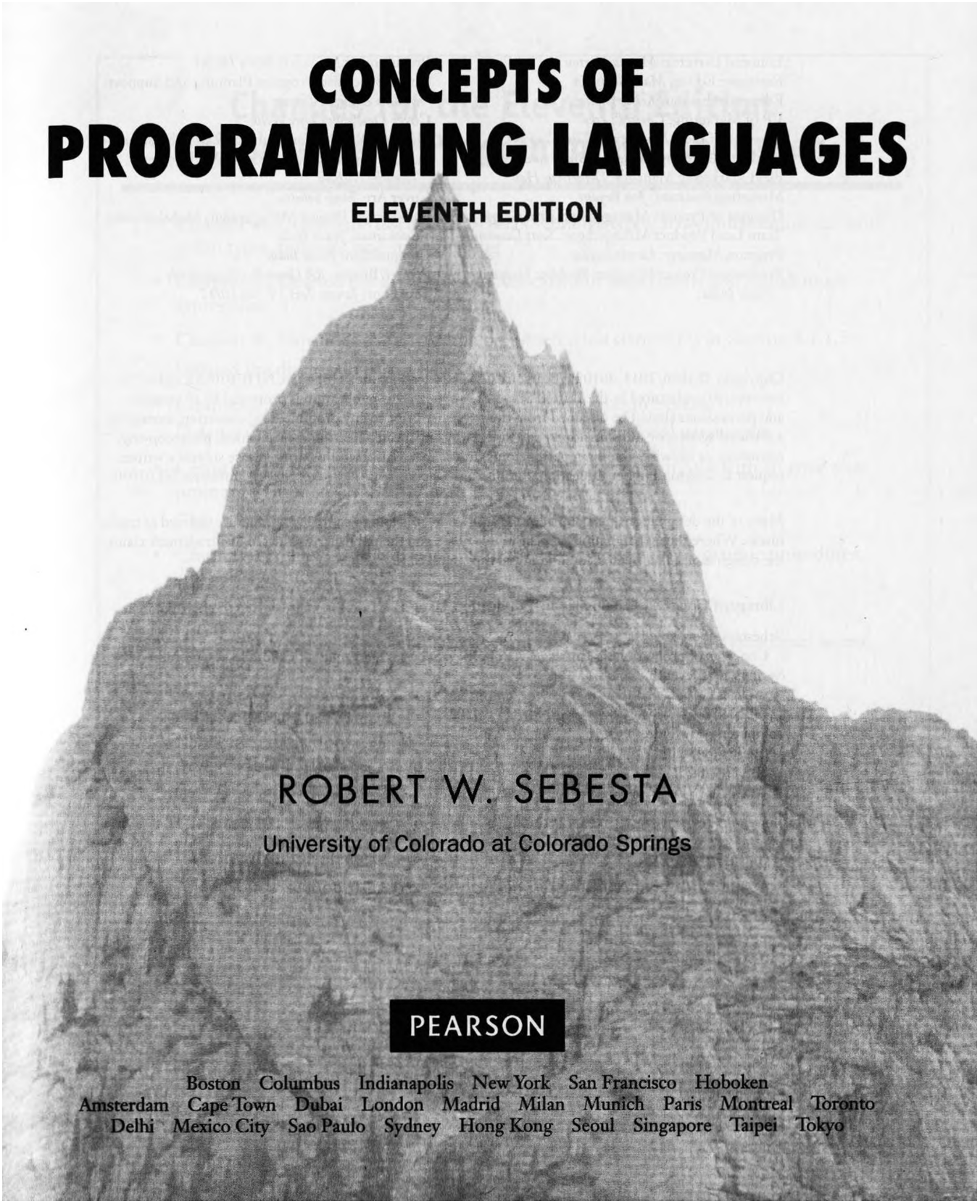
CONCEPTS OF PROGRAMMING LANGUAGES

ELEVENTH EDITION



ROBERT W. SEBESTA

University of Colorado at Colorado Springs

PEARSON

Boston Columbus Indianapolis New York San Francisco Hoboken
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto
Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Editorial Director: *Marcia Horton*
Executive Editor: *Matt Goldstein*
Editorial Assistant: *Kelsey Loanes*
VP of Marketing: *Christy Lesko*
Director of Field Marketing: *Tim Galligan*
Product Marketing Manager: *Bram van Kempen*
Field Marketing Manager: *Demetrius Hall*
Marketing Assistant: *Jon Bryant*
Director of Product Management: *Erin Gregg*
Team Lead Product Management: *Scott Disanno*
Program Manager: *Carole Snyder*
Production Project Manager: *Pavithra Jayapaul, Jouve India*
Procurement Manager: *Mary Fischer*
Senior Specialist, Program Planning and Support: *Maura Zaldivar-Garcia*
Cover Designer: *Joyce Wells*
Manager, Rights Management: *Rachel Youdelman*
Senior Project Manager, Rights Management: *Timothy Nicholls*
Cover Art: *Jacob Sebesta*
Full-Service Project Management: *Mahalatchoumy Saravanan, Jouve India*
Composition: *Jouve India*
Printer/Binder: *RR Donnelley Kendallville*
Text Font: *Janson Text LT Std 10/12*

Library of Congress Cataloging-in-Publication Data

Sebesta, Robert W.
Concepts of programming languages / Robert W. Sebesta, University of Colorado at Colorado Springs. — Eleventh edition.
pages cm
ISBN 978-0-13-394302-3 — ISBN 0-13-394302-X 1. Programming languages (Electronic computers) I. Title.
QA76.7.S43 2015
005.13—dc23 2014045178

2 16

PEARSON

ISBN 10: 0-13-394302-X
ISBN 13: 978-0-13-394302-3

The objectives of this chapter are to introduce the concepts of logic programming and logic programming languages, including a brief description of a subset of Prolog. We begin with an introduction to predicate calculus, which is the basis for logic programming languages. This is followed by a discussion of how predicate calculus can be used for automatic theorem-proving systems. Then, we present a general overview of logic programming. Next, a lengthy section introduces the basics of the Prolog programming language, including arithmetic, list processing, and a trace tool that can be used to help debug programs and also to illustrate how the Prolog system works. The final two sections describe some of the problems of Prolog as a logic language and some of the application areas in which Prolog has been used.

16.1 Introduction

Chapter 15 discusses the functional programming paradigm, which is significantly different from the software development methodologies used with the imperative languages. In this chapter, we describe another different programming methodology. In this case, the approach is to express programs in a form of symbolic logic and use a logical inferencing process to produce results. Logic programs are declarative rather than procedural, which means that only the specifications of the desired results are stated rather than detailed procedures for producing them. Programs in logic programming languages are collections of facts and rules. Such a program is used by asking it questions, which it attempts to answer by consulting the facts and rules. "Consulting" is perhaps misleading here, for the process is far more complex than that word connotes. This approach to problem solving may sound like it addresses only a very narrow category of problems, but it is more flexible than might be thought.

Programming that uses a form of symbolic logic as a programming language is often called **logic programming**, and languages based on symbolic logic are called **logic programming languages**, or **declarative languages**. We have chosen to describe the logic programming language Prolog, because it is the only widely used logic language.

The syntax of logic programming languages is remarkably different from that of the imperative and functional languages. The semantics of logic programs also bears little resemblance to that of imperative-language programs. These observations should lead the reader to some curiosity about the nature of logic programming and declarative languages.

16.2 A Brief Introduction to Predicate Calculus

Before we can discuss logic programming, we must briefly investigate its basis, which is formal logic. This is not our first contact with formal logic in this book; it was used extensively in the axiomatic semantics described in Chapter 3.

A **proposition** can be thought of as a logical statement that may or may not be true. It consists of objects and the relationships among objects. Formal logic was developed to provide a method for describing propositions, with the goal of allowing those formally stated propositions to be checked for validity.

Symbolic logic can be used for the three basic needs of formal logic: to express propositions, to express the relationships between propositions, and to describe how new propositions can be inferred from other propositions that are assumed to be true.

There is a close relationship between formal logic and mathematics. In fact, much of mathematics can be thought of in terms of logic. The fundamental axioms of number and set theory are the initial set of propositions, which are assumed to be true. Theorems are the additional propositions that can be inferred from the initial set.

The particular form of symbolic logic that is used for logic programming is called **first-order predicate calculus** (though it is a bit imprecise, we will usually refer to it as *predicate calculus*). In the following subsections, we present a brief look at predicate calculus. Our goal is to lay the groundwork for a discussion of logic programming and the logic programming language Prolog.

16.2.1 Propositions

The objects in logic programming propositions are represented by simple terms, which are either constants or variables. A constant is a symbol that represents an object. A variable is a symbol that can represent different objects at different times, although in a sense that is far closer to mathematics than the variables in an imperative programming language.

The simplest propositions, which are called **atomic propositions**, consist of compound terms. A **compound term** is one element of a mathematical relation, written in a form that has the appearance of mathematical function notation. Recall from Chapter 15 that a mathematical function is a mapping, which can be represented either as an expression or as a table or list of tuples. Compound terms are elements of the tabular definition of a function.

A compound term is composed of two parts: a **functor**, which is the function symbol that names the relation, and an ordered list of parameters, which together represent an element of the relation. A compound term with a single parameter is a 1-tuple; one with two parameters is a 2-tuple, and so forth. For example, we might have the two propositions

```
man (jake)
like (bob, steak)
```

which state that {jake} is a 1-tuple in the relation named man, and that {bob, steak} is a 2-tuple in the relation named like. If we added the proposition

```
man (fred)
```

to the two previous propositions, then the relation man would have two distinct elements, {jake} and {fred}. All of the simple terms in these propositions—man, jake, like, bob, and steak—are constants. Note that these propositions have no intrinsic semantics. They mean whatever we want them to mean. For example, the second example may mean that bob likes steak, or that steak likes bob, or that bob is in some way similar to a steak.

Propositions can be stated in two modes: one in which the proposition is defined to be true, and one in which the truth of the proposition is something that is to be determined. In other words, propositions either can be facts or queries. The example propositions above could be either.

Compound propositions have two or more atomic propositions, which are connected by logical connectors, or operators, in the same way compound logic expressions are constructed in imperative languages. The names, symbols, and meanings of the predicate calculus logical connectors are as follows:

Name	*Symbol*	*Example*	*Meaning*
negation	$\neg$	$\neg$ a	not *a*
conjunction	$\cap$	*a* $\cap$ *b*	*a* and *b*
disjunction	$\cup$	*a* $\cup$ *b*	*a* or *b*
equivalence	$\equiv$	*a* $\equiv$ *b*	*a* is equivalent to *b*
implication	$\supset$	*a* $\supset$ *b*	*a* implies *b*
	$\subset$	*a* $\subset$ *b*	*b* implies *a*

The following are examples of compound propositions:

a $\cap$ b $\supset$ c
a $\cap$ $\neg$ b $\supset$ d

The $\neg$ operator has the highest precedence. The operators $\cap$, $\cup$, and $\equiv$ all have higher precedence than $\supset$ and $\subset$. So, the second example above is equivalent to

(a $\cap$ ($\neg$ b)) $\supset$ d

Variables can appear in propositions but only when introduced by special symbols called *quantifiers*. Predicate calculus includes two quantifiers, as described below, where *X* is a variable and *P* is a proposition:

Name	*Example*	*Meaning*
universal	$\forall X.P$	For all *X*, *P* is true.
existential	$\exists X.P$	There exists a value of *X* such that *P* is true.

The period between X and P simply separates the variable from the proposition. For example, consider the following:

$\forall X.$ (woman (X) $\supset$ human (X))
$\exists X.$ (mother (mary, X) $\cap$ male (X))

The first of these propositions means that for any value of X, if X is a woman, then X is a human. The second means that there exists a value of X such that mary is the mother of X and X is a male; in other words, mary has a son. The scope of the universal and existential quantifiers is the atomic propositions to which they are attached. This scope can be extended using parentheses, as in the two compound propositions just described. So, the universal and existential quantifiers have higher precedence than any of the operators.

16.2.2 Clausal Form

We are discussing predicate calculus because it is the basis for logic programming languages. As with other languages, logic languages are best in their simplest form, meaning that redundancy should be minimized.

One problem with predicate calculus as we have described it thus far is that there are too many different ways of stating propositions that have the same meaning; that is, there is a great deal of redundancy. This is not such a problem for logicians, but if predicate calculus is to be used in an automated (computerized) system, it is a serious problem. To simplify matters, a standard form for propositions is desirable. Clausal form, which is a relatively simple form of propositions, is one such standard form. All propositions can be expressed in clausal form. A proposition in clausal form has the following general syntax:

$$B_1 \cup B_2 \cup \ldots \cup B_n \subset A_1 \cap A_2 \cap \ldots \cap A_m$$

in which the A's and B's are terms. The meaning of this clausal form proposition is as follows: If all of the A's are true, then at least one B is true. The primary characteristics of clausal form propositions are the following: Existential quantifiers are not required; universal quantifiers are implicit in the use of variables in the atomic propositions; and no operators other than conjunction and disjunction are required. Also, conjunction and disjunction need appear only in the order shown in the general clausal form: disjunction on the left side and conjunction on the right side. All predicate calculus propositions can be algorithmically converted to clausal form. Nilsson (1971) gives proof that this can be done, as well as a simple conversion algorithm for doing it.

The right side of a clausal form proposition is called the **antecedent**. The left side is called the **consequent** because it is the consequence of the truth of the antecedent. As examples of clausal form propositions, consider the following:

likes (bob, trout) $\subset$ likes (bob, fish) $\cap$ fish (trout)

father (louis, al) ∪ father (louis, violet) ⊂
father (al, bob) ∩ mother (violet, bob) ∩ grandfather (louis, bob)

The English version of the first of these states that if bob likes fish and a trout is a fish, then bob likes trout. The second states that if al is bob's father and violet is bob's mother and louis is bob's grandfather, then louis is either al's father or violet's father.

16.3 Predicate Calculus and Proving Theorems

Predicate calculus provides a method of expressing collections of propositions. One use of collections of propositions is to determine whether any interesting or useful facts can be inferred from them. This is exactly analogous to the work of mathematicians, who strive to discover new theorems that can be inferred from known axioms and theorems.

The early days of computer science (the 1950s and early 1960s) saw a great deal of interest in automating the theorem-proving process. One of the most significant breakthroughs in automatic theorem proving was the discovery of the resolution principle by Alan Robinson (1965) at Syracuse University.

Resolution is an inference rule that allows inferred propositions to be computed from given propositions, thus providing a method with potential application to automatic theorem proving. Resolution was devised to be applied to propositions in clausal form. The concept of resolution is the following: Suppose there are two propositions with the forms

$P_1 \subset P_2$
$Q_1 \subset Q_2$

Their meaning is that P_2 implies P_1 and Q_2 implies Q_1. Furthermore, suppose that P_1 is identical to Q_2, so that we could rename P_1 and Q_2 as T. Then, we could rewrite the two propositions as

$T \subset P_2$
$Q_1 \subset T$

Now, because P_2 implies T and T implies Q_1, it is logically obvious that P_2 implies Q_1, which we could write as

$Q_1 \subset P_2$

The process of inferring this proposition from the original two propositions is resolution.

As another example, consider the two propositions:

older (joanne, jake) ⊂ mother (joanne, jake)
wiser (joanne, jake) ⊂ older (joanne, jake)

From these propositions, the following proposition can be constructed using resolution:

wiser (joanne, jake) ⊂ mother (joanne, jake)

The mechanics of this resolution construction are simple: The terms of the left sides of the two clausal propositions are OR'd together to make the left side of the new proposition. Then the right sides of the two clausal propositions are AND'd together to get the right side of the new proposition. Next, any term that appears on both sides of the new proposition is removed from both sides. The process is exactly the same when the propositions have multiple terms on either or both sides. The left side of the new inferred proposition initially contains all of the terms of the left sides of the two given propositions. The new right side is similarly constructed. Then the term that appears on both sides of the new proposition is removed. For example, if we have

father (bob, jake) ∪ mother (bob, jake) ⊂ parent (bob, jake)
grandfather (bob, fred) ⊂ father (bob, jake) ∩ father (jake, fred)

resolution says that

mother (bob, jake) ∪ grandfather (bob, fred) ⊂
parent (bob, jake) ∩ father (jake, fred)

which has all but one of the atomic propositions of both of the original propositions. The one atomic proposition that allowed the operation father (bob, jake) in the left side of the first and in the right side of the second is left out. In English, we would say

if: bob is the parent of jake implies that bob is either the father or mother of jake
and: bob is the father of jake and jake is the father of fred implies that bob is the grandfather of fred
then: *if* bob is the parent of jake and jake is the father of fred *then:* either bob is jake's mother or bob is fred's grandfather

Resolution is actually more complex than these simple examples illustrate. In particular, the presence of variables in propositions requires resolution to find values for those variables that allow the matching process to succeed. This process of determining useful values for variables is called **unification**. The temporary assigning of values to variables to allow unification is called **instantiation**.

It is common for the resolution process to instantiate a variable with a value, fail to complete the required matching, and then be required to backtrack and instantiate the variable with a different value. We will discuss unification and backtracking more extensively in the context of Prolog.

A critically important property of resolution is its ability to detect any inconsistency in a given set of propositions. This is based on the formal property of resolution called **refutation complete**. What this means is that given a set of inconsistent propositions, resolution can prove them to be inconsistent. This allows resolution to be used to prove theorems, which can be done as

follows: We can envision a theorem proof in terms of predicate calculus as a given set of pertinent propositions, with the negation of the theorem itself stated as a new proposition. The theorem is negated so that resolution can be used to prove the theorem by finding an inconsistency. This is proof by contradiction, a frequently used approach to proving theorems in mathematics. Typically, the original propositions are called the **hypotheses**, and the negation of the theorem is called the **goal**.

Theoretically, this process is valid and useful. The time required for resolution, however, can be a problem. Although resolution is a finite process when the set of propositions is finite, the time required to find an inconsistency in a large database of propositions may be huge.

Theorem proving is the basis for logic programming. Much of what is computed can be couched in the form of a list of given facts and relationships as hypotheses, and a goal to be inferred from the hypotheses, using resolution.

Resolution on a hypotheses and a goal that are general propositions, even if they are in clausal form, is often not practical. Although it may be possible to prove a theorem using clausal form propositions, it may not happen in a reasonable amount of time. One way to simplify the resolution process is to restrict the form of the propositions. One useful restriction is to require the propositions to be Horn clauses. **Horn clauses** only can be in one of two forms: They have either a single atomic proposition on the left side or an empty left side.[1] The left side of a clausal form proposition is sometimes called the head, and Horn clauses with left sides are called headed Horn clauses. Headed Horn clauses are used to state relationships, such as

likes (bob, trout) $\subset$ likes (bob, fish) $\cap$ fish (trout)

Horn clauses with empty left sides, which are often used to state facts, are called *headless Horn clauses*. For example,

father (bob, jake)

Most, but not all, propositions can be stated as Horn clauses. The restriction to Horn clauses makes resolution a practical process for proving theorems.

16.4 An Overview of Logic Programming

Languages used for logic programming are called *declarative languages*, because programs written in them consist of declarations rather than assignments and control flow statements. These declarations are actually statements, or propositions, in symbolic logic.

One of the essential characteristics of logic programming languages is their semantics, which is called **declarative semantics**. The basic concept of this semantics is that there is a simple way to determine the meaning of each statement, and it does not depend on how the statement might be used to solve a

1. Horn clauses are named after Alfred Horn (1951), who studied clauses in this form.

problem. Declarative semantics is considerably simpler than the semantics of the imperative languages. For example, the meaning of a given proposition in a logic programming language can be concisely determined from the statement itself. In an imperative language, the semantics of a simple assignment statement requires examination of local declarations, knowledge of the scoping rules of the language, and possibly even examination of programs in other files just to determine the types of the variables in the assignment statement. Then, assuming the expression of the assignment contains variables, the execution of the program prior to the assignment statement must be traced to determine the values of those variables. The resulting action of the statement, then, depends on its run-time context. Comparing this semantics with that of a proposition in a logic language, with no need to consider textual context or execution sequences, it is clear that declarative semantics is far simpler than the semantics of imperative languages. Thus, declarative semantics is often stated as one of the advantages that declarative languages have over imperative languages (Hogger, 1984, pp. 240–241).

Programming in both imperative and functional languages is primarily procedural, which means that the programmer knows *what* is to be accomplished by a program and instructs the computer on exactly *how* the computation is to be done. In other words, the computer is treated as a simple device that obeys orders. Everything that is computed must have every detail of that computation spelled out. Some believe that this is the essence of the difficulty of programming using imperative and functional languages.

Programming in a logic programming language is nonprocedural. Programs in such languages do not state exactly *how* a result is to be computed but rather describe the form of the result. The difference is that we assume the computer system can somehow determine *how* the result is to be computed. What is needed to provide this capability for logic programming languages is a concise means of supplying the computer with both the relevant information and a method of inference for computing desired results. Predicate calculus supplies the basic form of communication to the computer, and resolution provides the inference technique.

Sorting is commonly used to illustrate the difference between procedural and nonprocedural systems. In a language like Java, sorting is done by explaining in a Java program all of the details of some sorting algorithm to a computer that has a Java compiler. The computer, after translating the Java program into machine code or some interpretive intermediate code, follows the instructions and produces the sorted list.

In a nonprocedural language, it is necessary only to describe the characteristics of the sorted list: It is some permutation of the given list such that for each pair of adjacent elements, a given relationship holds between the two elements. To state this formally, suppose the list to be sorted is in an array named list that has a subscript range $1 \ldots n$. The concept of sorting the elements of the given list, named old_list, and placing them in a separate array, named new_list, can then be expressed as follows:

$$\text{sort}(\text{old\_list}, \text{new\_list}) \subset \text{permute}(\text{old\_list}, \text{new\_list}) \cap \text{sorted}(\text{new\_list})$$
$$\text{sorted}(\text{list}) \subset \forall j \text{ such that } 11 \leq j < n, \text{list}(j) \leq \text{list}(j + 1)$$

where permute is a predicate that returns true if its second parameter array is a permutation of its first parameter array.

From this description, the nonprocedural language system could produce the sorted list. That makes nonprocedural programming sound like the mere production of concise software requirements specifications, which is a fair assessment. Unfortunately, however, it is not that simple. Logic programs that use only resolution face serious problems of execution efficiency. In our example of sorting, if the list is long, the number of permutations is huge, and they must be generated and tested, one by one, until the one that is in order is found—a very lengthy process. Of course, one must consider the possibility that the best form of a logic language may not yet have been determined, and good methods of creating programs in logic programming languages for large problems have not yet been developed.

16.5 The Origins of Prolog

As was stated in Chapter 2, Alain Colmerauer and Phillippe Roussel at the University of Aix-Marseille, with some assistance from Robert Kowalski at the University of Edinburgh, developed the fundamental design of Prolog. Colmerauer and Roussel were interested in natural-language processing; Kowalski was interested in automated theorem proving. The collaboration between the University of Aix-Marseille and the University of Edinburgh continued until the mid-1970s. Since then, research on the development and use of the language has progressed independently at those two locations, resulting in, among other things, two syntactically different dialects of Prolog.

The development of Prolog and other research efforts in logic programming received limited attention outside of Edinburgh and Marseille until the announcement in 1981 that the Japanese government was launching a large research project called the Fifth Generation Computing Systems (FGCS; Fuchi, 1981; Moto-oka, 1981). One of the primary objectives of the project was to develop intelligent machines, and Prolog was chosen as the basis for this effort. The announcement of FGCS aroused a sudden strong interest in artificial intelligence and logic programming in researchers and the governments of the United States and several European countries.

After a decade of effort, the FGCS project was quietly dropped. Despite the great assumed potential of logic programming and Prolog, little of great significance had been discovered. This led to the decline in the interest in and use of Prolog, although it still has its proponents.

16.6 The Basic Elements of Prolog

There are now a number of different dialects of Prolog. These can be grouped into several categories: those that grew from the Marseille group, those that came from the Edinburgh group, and some dialects that have been developed